REPORT: VehicleVision Classification

Logistic Regression, Engineered Tree Hybrid, and Soft Voting

Machine Learning Course Project | 15 October 2026

This report describes the code and saved outputs in **vehicle_logistic.ipynb**, **vehicle_dt.ipynb**, and **vehicle_ensemble.ipynb**. The three implementations predict vehicle classes from numerical specifications and write submission CSV files.

1. Problem and Dataset

The task is supervised classification into three categories: **0 - Sedan, 1 - SUV, and 2 - Sports Car**. The target is **vehicle_class**. The notebooks exclude **vehicle_id** from model inputs and preserve it in the submission.

Input file	Rows	Columns	Use
vehicle_train_10000.csv	10,000	22	Measurements and labels
vehicle_test.csv	2,500	21	Measurements for prediction

The original representation has 20 input features. The standalone LR and simple ensemble use those original features. The tree-hybrid notebook adds 17 derived features, producing 37 predictors.

2. Dataset Inspection

The LR notebook displays the training shape, the first five rows, the target classes, and class counts. These are the dataset-inspection results recorded in the supplied notebooks.

Vehicle class	Training rows
Sedan (0)	3,389
SUV (1)	3,326
Sports Car (2)	3,285

The three counts are similar. The simple ensemble also prints **7,500 cleaned training rows** after removing duplicate feature-and-label rows, and **2,500 test rows**. Its cleaning step therefore removes 2,500 repeated rows from the original training file.

Accuracy measures the fraction of correctly predicted labels. The LR and simple ensemble notebooks also print precision, recall, F1, and support through classification_report.

Project GitHub: https://github.com/shubranshugupta/ml-project1

3. Preprocessing and Feature Engineering

3.1 Preprocessing used in each notebook

Implementation	Input preparation	Model preprocessing
Standalone LR	Exclude ID and target. Retain the 10,000 raw training rows.	Median imputation, followed by StandardScaler.
Tree hybrid	Create 17 features for both training and test data; exclude ID and target. Retain the raw training rows.	StandardScaler for its LR branch. No general imputer is included in this notebook.
Simple ensemble	Convert infinities to NaN, remove duplicate feature-and-label rows, and check the feature columns and target labels.	Separate median-imputation pipelines; add scaling only to LR.

Median imputation fills a missing numerical reading using a median learned from fitting rows. **StandardScaler** centers and scales features before LR. make_pipeline keeps these steps together with the classifier. The bagged trees in the simple ensemble use imputation without scaling.

3.2 Features added in vehicle_dt.ipynb

create_super_features() copies the input frame and adds the following feature families. The same function is applied to training and test measurements.

Family	Derived features	How they are used
Powertrain (6)	power_to_weight, torque_to_weight, battery_to_weight, battery_to_accel, power_per_cyl, torque_per_disp	Express power, torque, and battery readings relative to other specifications.
Geometry (4)	volume_est, density_est, wheelbase_to_length, length_to_width	Describe approximate body size, mass per estimated volume, and dimension ratios.
Aerodynamics (2)	frontal_area, drag_area	Combine width and height, then multiply by the drag coefficient.
Energy and handling (5)	kinetic_energy, braking_force, avg_fuel_efficiency, fuel_tank_to_weight, low_slung_ratio	Combine mass and braking distance, average fuel efficiency, and create capacity or clearance ratios.

For example, power_to_weight divides horsepower by weight. volume_est uses length x width x height with a unit conversion. kinetic_energy assumes 100 km/h, and braking_force divides that estimate by braking distance. These are derived numerical proxies, not additional measured attributes. Some ratios add 1e-5 to their denominator.

4. Models and Ensemble Methods

4.1 Logistic Regression

LR learns class scores from the supplied features and predicts a vehicle label. The standalone notebook and simple ensemble use **C=1.0, max_iter=3000, solver="lbfgs"**, with median imputation and scaling. C controls regularization strength, while max_iter caps optimization iterations.

4.2 Engineered tree hybrid in vehicle_dt.ipynb

Despite its filename, this notebook trains four tree ensembles and an LR model on all **10,000 rows and 37 features**. Its settings are:

Branch	Explicit settings
Gini bagging	300 trees; maximum depth 16; minimum leaf size 2; feature fraction 0.85; seed 42.
Entropy bagging	300 trees; entropy criterion; maximum depth 16; minimum leaf size 2; feature fraction 0.85; seed 101.
Random Forest	500 trees; minimum split size 4; seed 42.
Extra Trees	500 trees; minimum split size 4; seed 42.
LR branch	StandardScaler; C=1.0; maximum iterations 2,000; seed 42.

Bagging averages trees fitted to resampled data. **Random Forest** also samples features during split selection. **Extra Trees** adds random split thresholds. Gini and entropy are two split criteria. The tree branches contain **1,600 trees** in total.

4.3 Probability averaging in the two implementations

The engineered hybrid gives each tree branch one quarter of the tree probability average, then combines that average with its LR probabilities:

```
P_tree = (P_gini + P_entropy + P_RF + P_ET) / 4
P_hybrid = 0.40 * P_tree + 0.60 * P_LR
prediction = argmax(P_hybrid)
```

Thus each tree branch contributes 10% and LR contributes 60% to the final hybrid probabilities. The weights are assigned directly in the code.

The **simple ensemble** instead uses VotingClassifier(voting="soft") with **95% LR + 5% bagging**. Its bagging model has 80 trees, unrestricted maximum depth, minimum leaf size 5, sample fraction 0.8, feature fraction 1.0, bootstrap sampling, and seed 42. It uses the original 20 features, not the 37-feature hybrid.

Both approaches choose the class with the largest combined probability. The supplied notebooks do not run a weight search.

5. Validation Results and Discussion

5.1 How validation is performed

Standalone LR: train_test_split uses a 20% validation fraction, seed 42, and stratify=y. It fits on 8,000 raw rows and evaluates on 2,000. Stratification helps retain the class proportions in both splits.

Simple ensemble: identical measurement vectors form groups. StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=2026) supplies splits, but the code takes **only the first split**. It is a single grouped 80/20 validation check, not an average of five CV runs. The same validation rows are used for LR, bagging, and their ensemble; the weights remain fixed.

5.2 Saved accuracy results

Model / implementation	Accuracy	Evaluation recorded
Standalone LR	88.35%	2,000 raw validation rows.
LR inside simple ensemble	87.73%	Grouped split of cleaned data.
Bagging inside simple ensemble	87.13%	Same grouped validation rows.
95% LR + 5% bagging	87.67%	Same grouped validation rows.
37-feature engineered hybrid	Not reported	Fits all training rows and predicts test data; no validation step.

The 88.35% and 87.73% LR results use different training data, split procedures, and validation rows. They do not isolate the effect of cleaning. On the common cleaned split, LR is slightly stronger than bagging and the 95/5 blend; this run does not show an ensemble accuracy gain.

5.3 Printed class report for the simple ensemble

Vehicle class	Precision	Recall	F1	Support
Sedan	0.83	0.83	0.83	521
SUV	0.88	0.91	0.89	495
Sports Car	0.93	0.90	0.92	484

Values above use the notebook's displayed two-decimal precision. SUV has the highest displayed recall, and Sports Car has the highest precision. The class supports sum to 1,500 validation vehicles.

The hybrid notebook prints predicted test-class shares: SUV 33.36%, Sedan 33.32%, and Sports Car 33.32%. These describe its prediction distribution and do not measure correctness. No validation score is saved for that hybrid.

6. Model Choice and Submission Generation

6.1 Best supported result

Within the simple ensemble's common validation split, **LR gives the highest accuracy: 87.73%**, followed by the blend at 87.67% and bagging at 87.13%. LR is therefore the stronger baseline in that direct comparison. The engineered hybrid cannot be ranked against them from these notebooks because it has no recorded validation score.

The standalone LR notebook records 88.35% on its own split, but that does not establish superiority under the cleaned grouped protocol. The notebooks contain no test labels or recorded leaderboard scores, so this report makes no claim about hidden-test performance.

6.2 Final training and output files

Implementation	Final fitting data	Submission file
Standalone LR	All 10,000 raw rows; 20 features.	vehicle_submission.csv
Engineered hybrid	All 10,000 raw rows; 37 features for all five branches.	submission_hybrid_super_blend.csv
Simple ensemble	All 7,500 cleaned rows; 20 features and fixed 95/5 weights.	submission_simple_lr_bagging.csv

Every submission contains **vehicle_id** and an integer **vehicle_class**. The test file has 2,500 rows. ID values come from the test data, and predictions are generated in the same row order. The standalone LR refits its pipeline after validation and uses Google Colab's files.download() to download its saved CSV.

The engineered hybrid fits its models directly on the full training data, calculates the combined probability vector, and saves the class with the largest probability. The simple ensemble refits VotingClassifier on all cleaned rows after validation, then writes its predictions.

6.3 Checks implemented in the simple ensemble

Assertions check that test feature columns match training inputs, target labels are nonmissing and belong to {0, 1, 2}, and training/validation measurement groups do not overlap. Before saving, the notebook checks the submission row count, unique IDs, absence of missing values, valid predicted classes, and original test-ID order.

The other two notebooks display a submission preview or predicted class counts. They do not contain the same explicit submission assertions, so those stronger checks are attributed only to the simple ensemble.

6.4 Conclusion

The three notebooks implement a regularized LR baseline, a feature-engineered hybrid of four tree ensembles and LR, and a simpler fixed-weight LR/bagging vote. Their saved outputs support the reported local validation scores and successful CSV generation. The simple blend does not improve on its LR component in the recorded comparison; the larger hybrid needs its own validation before a performance advantage can be claimed.

Flight Price Prediction

Project Report | 15 October 2026

The project predicts ticket prices using a linear-regression pipeline, Random Forest, Extra Trees, and combinations of their predictions. The submitted ensemble uses an equal geometric average of LR and the RF + ET blend.

Reported public leaderboard R^2: 0.96181. Local results below are taken from the saved notebook outputs; the public score is the reported submission result.

1. Problem and Dataset

This is a supervised regression task: learn from tickets with known prices and predict the price of unseen tickets. Inputs describe the carrier, flight, cities, time categories, stops, cabin class, duration, and days left before departure. The ID is retained for submission and excluded from model predictors.

Dataset	Rows	Columns	Target
Training	15,589	12	Price in rupees
Test	3,898	11	Not supplied

2. Exploratory Data Analysis

Check	Saved finding	Why it matters
Missing values	Airline: 3,089 train / 799 test. Duration: 3,095 train / 776 test.	Fill missing information instead of discarding many rows.
Category counts	1,070 flight codes; 18 airlines; 6 source and 6 destination cities; 2 cabin classes.	Represent categories and related flight information carefully.
Numerical ranges	Duration: 0.92-38.08 hours in train; 0.92-29.08 in test. Days left: 1-57 in both.	Observed test ranges fall within training ranges.

The notebook plots ticket-price distribution, price by cabin class, and price against days left. These checks motivate log-price training for forests, cabin-specific LR models, and booking-time features. They do not establish a fixed causal effect of booking earlier.

3. Data Preprocessing and Feature Engineering

3.1 Missing values and encoding

Step	RF + Extra Trees	LR pipeline
Carrier	Recover missing airline from flight-prefix mappings.	Use flight prefix directly as the carrier input.
Duration	Try flight/route median, flight median, route/stops median, then global median.	Use source/destination/stops median, then global median; add a missing-duration flag.
Categories	Target-encode 11 categorical features; represent cabin, stops and time categories numerically too.	One-hot encode carrier, cities, times and stops.
Scale and target	No feature scaling; train on log(1 + price).	Scale numerical and interaction inputs; predict raw prices.

Target encoding replaces a category with a smoothed price-related estimate. Its training encodings are internally cross-fitted, and validation/test encodings use the fold-training labels. One-hot encoding instead creates indicator columns for categories.

3.2 The 32 tree inputs

Feature group	What was used and why
Carrier and route	Prefix, airline, cities, route, airline/class, route/class, airline/route and route/stops: describe different market and cabin combinations.
Schedule and trip	Departure/arrival categories and ordered codes, cabin, stops, duration and direct-flight flag: represent the journey.
Booking horizon	Days left, log and inverse days left, and indicators for <=2, <=5 and >=25 days: expose different booking windows.
Relative duration	Minimum route duration, excess duration, duration per stop and a duration ratio: summarize journey efficiency.
Frequency	Counts for flight, airline, route and airline/route: describe how often a pattern occurs in the training data.

The final matrix contains 11 categorical and 21 numerical inputs. Flight identity supports duration recovery and counts, but is not directly target-encoded. Duration ratios and excess duration are proxies, not measured speed or connection time; counts describe dataset frequency, not measured demand.

Some airline and duration mappings use combined training/application features, while frequency counts use all supplied training rows. No application prices are used, but this preprocessing is not fully isolated by validation fold.

3. Data Preprocessing and Feature Engineering

3.3 LR features and fitting stages

The LR pipeline uses six categorical inputs and five numerical inputs: duration, days left, log days left, a last-three-days indicator, and duration-missing status. It then adds the following stages to capture pricing patterns:

Stage	How it is used
Segment models	Fit Economy separately from Business with prefixes 6E/AI/IX and other Business carriers. These are carrier-based groups, not verified geographic classifications.
Comparable fares	Calculate median, lower quartile, minimum and mean prices from matching training itineraries. If needed, fall back to flight/route, flight, carrier/route, then the segment.
Interactions and Ridge	One-hot encode categories, scale numerical features, create pairwise interactions, scale again, and fit Ridge regression. Interactions allow two features to affect the price together.
Residual corrections	Fit smaller Ridge models to remaining prediction errors using flight and progressively more specific route/cabin/days-left keys. Add these corrections to the initial prediction.
Four-variant average	Average four versions with different fare-statistic inputs, penalties and correction stages to reduce reliance on one specification.
Nearby-day estimates	Average comparable training fares with weight exp(-absolute days-left difference / 5). Similar booking horizons receive more weight.
Nonnegative stack	Combine the four-variant prediction and the two nearby-day estimates using nonnegative least squares (NNLS), separately by segment and itinerary-peer availability.

Grouped fare statistics exclude the current training row where matching peers exist. The final segment-wide fallback includes all segment prices. Nearby-day estimates refer to days remaining before departure, not calendar dates.

3.4 Prediction checks

Each LR variant is clipped to its segment training minimum and 1.1 times its segment maximum before averaging. The forest test prediction is clipped to the training price range. Submission functions check row count, finite positive predictions, and unique IDs, then write the IDs with predicted prices.

4. Models and Evaluation

4.1 Models in simple terms

Model	How it works and how it is used
Random Forest (RF)	Averages decision trees trained on bootstrap samples, using a random subset of features at each split. Averaging reduces dependence on one tree.
Extra Trees (ET)	Averages trees with additional randomness in their split thresholds. The notebook uses its default of no bootstrap sampling.
DT pipeline: RF + ET	An equal blend of RF and ET predictions in log-price space, converted back to rupees. There is no separately fitted single-tree model.
Main LR pipeline	Ridge is linear regression with a penalty on large coefficients. Here it is combined with the feature and correction stages described in Section 3.
Final ensemble	Combines the LR and RF + ET price predictions. The exported method is their equal geometric average.

4.2 Settings and validation

Each forest uses **180 trees**, maximum depth **25**, minimum split size **6**, minimum leaf size **2**, and feature fraction **0.75**. Five shuffled folds use seed 42; model seeds are 42 plus the fold index. These are fixed settings in the supplied notebooks, not a documented hyperparameter search.

Forest validation predictions are out-of-fold (OOF): each row is predicted by a model that did not train on its price. Test predictions average the five fold models in price space. The LR notebook defines five-fold development evaluation and a separate 80/20 holdout; it has no saved numerical outputs for those checks.

The ensemble comparison uses **9,353 training, 3,118 calibration, and 3,118 evaluation rows**. Calibration learns combination weights. Evaluation labels are removed before predicting and are used only to calculate scores. Internal LR stacking weights are learned from OOF predictions within the supplied training portion.

RMSE emphasizes large errors; MAE is the average absolute price error; R^2 measures improvement over the evaluation-set mean predictor. Lower RMSE/MAE and higher R^2 are preferred. Scores from different validation protocols are not directly comparable.

The LR development stack is not fully nested cross-validation: base models producing other-fold stacking inputs may have seen the held-out fold. This qualification applies to that development calculation, rather than making its score an independent final-test estimate.

4. Models and Evaluation

4.3 Recorded model scores

Model	RMSE	MAE	R^2	Protocol
Main LR	9,682.5590	3,447.2779	0.9335	Ensemble evaluation
Extra Trees	10,329.7524	3,141.5442	0.9303	Five-fold OOF
DT: RF + ET	10,312.5195	3,154.7929	0.9305	Five-fold OOF
Random Forest	10,425.3633	3,207.6640	0.9290	Five-fold OOF
Geometric ensemble	9,537.6565	3,203.7948	0.9354	Ensemble evaluation

4.4 Ensemble techniques used

Method	Brief explanation
Arithmetic 50/50	Average LR and RF + ET prices directly.
Geometric 50/50	Take the square root of the product of the two positive prices.
Weighted average	Fit one LR weight between 0 and 1 on calibration data; the tree weight is the remainder.
Segment-weighted average	Fit a separate weight for each cabin/carrier segment; use the global weight if fewer than 100 calibration rows are available.
NNLS stacking	Learn nonnegative coefficients for both predictions; coefficients do not have to sum to one.
Ridge stacking	Fit a regularized regression with an intercept to the two prediction columns; scale them to thousands of rupees and use alpha 10.

The fitted global LR weight is **0.3128**, leaving approximately **0.6872** for RF + ET. This learned combination is evaluated alongside the fixed equal blends; it is not the method used in the final export.

4.5 Public leaderboard result

The reported public R^2 for the ensemble submission is **0.96181**. This result was supplied separately from the notebooks. No standalone LR, RF, ET or RF + ET public scores are reported here because the current notebooks do not record them.

5. Results and Discussion

The following methods were evaluated on the same 3,118 rows in the ensemble notebook. This gives a direct comparison between the two component pipelines and their combinations.

Method	RMSE	MAE	R^2
Arithmetic 50/50	9,532.9230	3,207.8193	0.9355
Geometric 50/50	9,537.6565	3,203.7948	0.9354
Weighted average	9,608.1868	3,159.4860	0.9345
NNLS stacking	9,639.0888	3,116.0775	0.9341
Segment-weighted	9,640.5957	3,169.3990	0.9340
Ridge stacking	9,644.4166	3,182.1596	0.9340
LR only	9,682.5590	3,447.2779	0.9335
RF + ET only	9,888.5784	3,175.7278	0.9306

Combining models helped. Both equal-weight blends improved RMSE and R^2 over LR or RF + ET alone. The arithmetic blend had the lowest local RMSE, beating the geometric blend by only about 4.73 rupees. NNLS had the lowest MAE but a worse RMSE.

The exported model uses geometric averaging. The notebook sets METHOD to Geometric 50/50, fits the component pipelines using the full training set, and writes 3,898 predictions to data/submission_ensemble_simple.csv. Its formula is:

Final price = square root of (LR prediction × RF + ET prediction)

The reported public score is $R^2 = 0.96181$. It is separate from the local geometric-blend R^2 of 0.9354 because the two scores use different data. The saved local results support blending, but do not show that geometric averaging is better than arithmetic averaging on the evaluation set.

The report reflects the implemented methods and saved outputs of the three current notebooks. The LR notebook defines additional evaluation steps without saved results, so no historical LR development scores are substituted for them.

Project Repository

The GitHub repository below brings together both projects presented in this report: VehicleVision Classification and Flight Price Prediction. It provides a common reference for exploring the implementation of each project alongside the methods and results discussed in the report.

GitHub repository

https://github.com/shubranshugupta/ml-project1